

implementation ->
verification コンサルティング
結果

2026-04-22

implementation -> verification コンサルティング結果

対象

実装成果物とテスト資産から検証工程文書や確認観点を整備する工程を対象とする。

要約

- AI は実装からテスト観点、確認手順、異常系観点を抽出する支援に向く。
- 人は品質基準、受け入れ判断、実機確認、失敗時の切り分けを担う。
- 自動化は高いが、検証結果の確定と出荷可否判断は人が保持する。

入力

- 実装成果物
- 詳細設計書
- 要件定義書
- テスト観点や受け入れ条件
- 既存または実装工程で追加・更新した `QTest` / `Squish` スクリプト
- 検証テンプレート
- 既知不具合一覧
- ビルドログ
- 実行ログ

出力

- 検証観点一覧
- テスト手順のたたき台
- 不足テストコードの追加対象一覧
- 想定不具合一覧
- 検証結果文書の草案

AI の役割

- 実装と設計を照合して確認項目を作る
- 正常系、異常系、境界値の観点を列挙する
- 操作手順を再現性のある形へ整理する
- 失敗時の確認ポイントを網羅的に挙げる

人の役割

- 重要度と優先度を決める
- 実際の動作確認を行う
- 不具合の再現確認と切り分けを行う
- 受け入れ可否を最終判断する

適用条件

- 実装成果物と設計差分を追える
- 実行環境と対象 OS がそろっている
- テスト保守担当または品質判断者がいる

非適用条件

- 実機確認の環境がなく、ログだけで受け入れ判断する
- GUI シナリオが未確定のまま回帰自動化へ進む

自動化できる範囲

- テスト観点抽出
- 手順書の下書き生成
- 実装差分に応じた回帰観点整理
- 検証結果文書の整形

自動化しにくい範囲

- 実機や環境依存の不具合判定
- 操作感や見た目の最終品質判断
- 不具合修正の妥当性承認

Qt 固有の重点論点

- Windows、macOS、Linux の表示差
- High DPI、フォント差、文字切れ、IME
- フォーカス遷移、入力デバイス差、マルチディスプレイ
- 再描画タイミング、ちらつき、透過表示
- `QSettings` の保存と再起動後の復元
- `signal / slot` とイベント駆動処理のタイミング差

推奨 LLM / 使い分け

- レビュー能力の高いモデル
 - 観点抽出と網羅確認に使う
- 長文処理モデル
 - 設計と実装の突き合わせに使う
- 低コストモデル
 - 手順整形と文書化に使う

AI と既存自動化手段の役割分担

- AI
 - テスト観点抽出、手順書草案、失敗ログ要約、再現条件整理を担う
- `GitHub Copilot plugin for Qt Creator`
 - テストコードや補助コードの入力補完を補助する
 - ただし、検証観点の妥当性そのものは保証しない
- `QTest`
 - ロジック、Widget、`signal / slot` の単体寄り検証を担う
- `Squish for Qt`
 - GUI 回帰、設定ダイアログ、操作シナリオの受け入れ寄り確認を担う
- `Squish AI Assistant`
 - `Squish` スクリプト改善、失敗解析、`runner / server` ログ読解を補助する
- `Qt Creator MCP server`

- `build`、`debug`、`list_issues` を通じたローカル再現と切り分けを補助する
 - ただし、現時点では新しく基本機能が中心のため、検証基盤の中心には置かず補助的に使う
 - 将来、取得できる文脈や操作対象が広がれば、切り分け支援の価値はさらに高まる
- `CTest` / `Sanitizers`
 - 決定論的なテスト実行と実行時異常検出を担う
- 人
 - 重要シナリオ選定、実機確認、受け入れ判断を担う

プロンプト設計方針

- 要件、設計、実装差分を同時に渡す
- 正常系、異常系、境界値を分けて出力させる
- 実施者が判断すべき項目を別枠に分ける
- 未検証項目を必ず残させる
- Squish の失敗ログや `Qt Creator` の `Issues` を再入力して、原因候補と追加確認項目を分けて出させる

テスト工程の整理

- `design`
 - テスト観点、受け入れ条件、重要シナリオを定義する
- `implementation`
 - `QTest` や `Squish` スクリプトなどのテストコードを追加・更新する
- `verification`
 - 実装工程でそろえた実装成果物とテスト資産を使って、テスト実行、結果記録、不具合起票、受け入れ判断を行う

導入段階

- PoC
 - `QTest` で単体寄りの観点を 1 件、`Squish for Qt` で主要 GUI シナリオを 1 本、それぞれ小さく自動化する
- 限定導入
 - `Squish AI Assistant` とログ解析を含めた保守運用を試す
- 本格導入
 - `QTest`、`Squish for Qt`、`Qt Creator MCP server`、CI の役割分担を標準化する

KPI

- テスト観点漏れ件数
- GUI 回帰失敗の解析時間
- `Squish` スクリプト保守時間
- OS 差分不具合件数
- 再現不能不具合率

顧客提案時の訴求点

- テスト観点の抜け漏れを減らせる
- 実装変更に従った検証整理がしやすい
- 人は重要な確認と受け入れ判断に集中できる